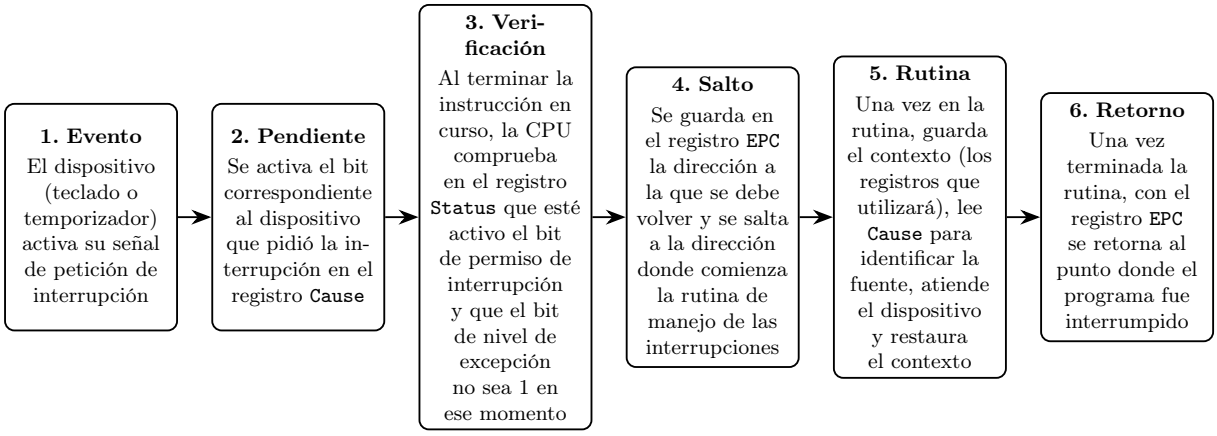
Informe de Arquitectura de Computadores
Interrupciones y Excepciones en MIPS32

Daniel Rodulfo 31725032

26 de octubre de 2026

1. Explique con un diagrama cómo funciona el ciclo de una interrupción de hardware (desde que ocurre el evento hasta que se atiende).

   Respuesta:



2. ¿Qué diferencias hay entre gestionar E/S con sondeo y hacerlo con interrupciones?

   Respuesta: La principal diferencia vendría siendo la revisión de los registros de E/S: con sondeo se tendría que consultar constantemente en un bucle el estado de los registros para saber cuándo se va a realizar una operación, mientras que con las interrupciones, en cuanto los registros detecten que se está realizando una petición de E/S, avisarían al sistema para generar una interrupción y así atender la petición.

3. ¿Qué ventajas tiene el uso de interrupciones en términos de uso del procesador?

   Respuesta: Permite que el procesador pueda realizar otras tareas en lugar de quedarse esperando en un ciclo de sondeo a que ocurra un evento de escritura o de lectura, facilitando la multitarea.

4. ¿Qué registros especiales se utilizan en MIPS32 para gestionar interrupciones?

Respuesta: Se utilizan los registros del coprocesador Status($12) para las máscaras de interrupciones y bits de autorización, Cause($13) para llevar un registro de la causa de las interrupciones y los bits de las interrupciones pendientes, EPC($14) para guardar la dirección de retorno para luego de que se atienda la interrupción, BadVAddr($8) para guardar la dirección de memoria en que ocurrió un problema de referencia de memoria, Count($9) como contador y temporizador y Compare($11), que vendría siendo el valor con el que se compara Count; causa una interrupción cuando coinciden.

5. ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio?

   Respuesta: Cuando se produce una interrupción, lo que estuviese ejecutándose queda en pausa hasta que se atienda la interrupción. Si no se resguardan los registros, en caso de que estos estuviesen siendo usados, esto conllevaría a que tuviesen valores incorrectos, provocando resultados o comportamientos que no corresponderían en una ejecución normal.

6. Momentos en que pueden generarse excepciones en un sistema MIPS32.

   a) Enumera al menos 4 situaciones en las que se pueda generar una excepción (por ejemplo: desbordamiento aritmético, fallo de dirección, etc.).
   b) Explica qué etapas del pipeline pueden provocar una excepción y por qué.

   Respuesta:

   A-)

   1- Desbordamiento aritmético (puede ser causado por instrucciones como add, sub, addi).

   2- Dirección desalineada o fuera de rango en carga o guardado en memoria (intento de acceder a una dirección inválida).

   3- Intentar ejecutar una instrucción reservada o ilegal (una instrucción cuyo código no corresponde a ninguna instrucción válida).

   4- Excepción de punto flotante.

   B-) IF: por un fallo al buscar la instrucción (dirección inválida). ID: por la ejecución de una instrucción no definida. EX: por un desbordamiento aritmético. MEM: por un acceso a una dirección de datos inválida o desalineada.

7. Estrategias de tratamiento de excepciones e interrupciones

   a) Explica las diferencias entre interrupciones y excepciones (¿son síncronas o asíncronas?).
   b) Describe brevemente dos estrategias para tratar excepciones en un sistema MIPS32. ¿Cómo se redirige la ejecución hacia la rutina de servicio? ¿Cuál es la función del registro EPC (Exception Program Counter)?

Respuesta:

A-) Las interrupciones son generadas por hardware externo y son asíncronas, ya que pueden suceder en cualquier momento y no tienen que coordinarse con ninguna instrucción, mientras que las excepciones son causadas por la misma instrucción y se consideran síncronas, ya que siempre suceden con la misma o mismas instrucciones.

B-) Una manera es transferir el control al sistema operativo en una dirección fija, donde, dependiendo de la causa de la excepción, se escoge cómo proceder. La otra manera es mediante interrupciones vectorizadas: cuando se produce la excepción se determina la causa y se transfiere el control directamente a la rutina específica para ese tipo de excepción. En ambos casos se carga en el PC la dirección de la rutina de servicio y se guarda en el EPC la dirección de la instrucción afectada o a la que se debe regresar una vez termine la rutina.

8. Habilitación de interrupciones en dispositivos y procesador

   a) Explica cómo se habilitan las interrupciones en:
      - El teclado.
      - La pantalla.
      - El procesador (detalla qué bits del registro Status deben modificarse).

   b) ¿Qué pasaría si habilitamos interrupciones en los dispositivos, pero no en el procesador?

Respuesta:

A-) Para el teclado se pone en 1 el bit 1 del registro de control del receptor (0xFFFF0000). Con eso el hardware genera una interrupción cuando llega una tecla. Para la pantalla se pone en 1 el bit 1 del registro de control del transmisor (0xFFFF0008); se interrumpe cuando está lista para recibir otro carácter. Para el procesador se modifican ciertos bits del registro Status: el bit 0 siempre se debe poner en 1 para la habilitación global de interrupciones y, dependiendo de dónde puedan provenir las interrupciones, se habilita uno o varios de los bits 8-15.

B-) Los dispositivos mandarían la petición de interrupción, pero el procesador no la aceptaría, por lo que sería ignorada. Esto generaría que los dispositivos no funcionasen de la forma en que se espera.

9. Procesamiento de interrupciones

   a) Describe paso a paso qué ocurre cuando se produce una interrupción de reloj:
      - Desde que el evento ocurre hasta que la rutina de servicio termina.
      - ¿Qué registros se guardan y restauran?
      - ¿Qué hace el hardware y qué hace el software (sistema operativo o rutina)?

   b) ¿Por qué es importante guardar el contexto (registros generales, EPC, Status) al entrar en la rutina?

Respuesta:

A-) Cuando el registro Count iguala a Compare, se activa en Cause el bit de interrupción de hardware. Luego se verifica en Status que no se esté atendiendo otra excepción; si no es el caso, se indica que ahora se está atendiendo una poniendo el bit 1 (nivel de excepción) de Status en 1, se guarda el PC en EPC y se salta a la dirección de la rutina de excepción. Una vez en la rutina se preservan los registros que se usarán, se lee el registro Cause para verificar cuál es la causa, se atiende la petición, se cambia el valor de Compare, se restauran los registros y, usando el EPC, se restaura el PC en donde había quedado.

B-) La idea de una interrupción es atender la petición requerida sin afectar el curso de las instrucciones que se estuviesen ejecutando. Si se usasen registros durante la rutina de atención y estos no se preservasen, cuando se regrese el control al flujo normal (usando el registro EPC para retornar a la instrucción que correspondía cuando se activó la interrupción) es posible que los valores de los registros que usaba el programa no sean los correctos, provocando así una corrupción de los datos; por esto se preservan. También puede ser importante preservar Status y EPC, ya que pueden haber interrupciones anidadas.

10. Interrupciones de reloj y control de ejecución

    a) Explica cómo una interrupción de reloj puede usarse para:
        - Evitar que un programa quede en un bucle infinito.
        - Finalizar programas que superan un tiempo máximo de ejecución.

    b) ¿Qué debe hacer el software si el programa finaliza antes de que ocurra la interrupción de reloj?

    Respuesta:

    A-) Una interrupción de reloj se puede ejecutar durante un bucle infinito y forzar la salida del mismo (quizás cambiando alguna bandera del bucle), funcionando como un límite de tiempo para la realización de una tarea o la espera de un evento. Bajo un principio similar, podría detener la ejecución de un programa que se haya pasado del tiempo establecido: una vez que se alcance el límite de tiempo establecido en Compare, dentro de la rutina de servicio se podría finalizar por completo la ejecución del programa haciendo syscall 10.

    B-) Si el programa termina antes, el software debe cancelar el temporizador: deshabilitar la interrupción de reloj, limpiar Compare o la interrupción pendiente y restaurar Status. Si no, la interrupción podría dispararse después, cuando ya no haya programa, y matar un proceso que no corresponde o ejecutar código inválido.

11. Análisis y Discusión de los Resultados

    Respuesta: Para ambos ejercicios se encontró cierta limitante para usar el registro de reloj Count, ya que MARS no lo tiene implementado. Para suplir esto se usó la llamada al sistema nro. 30, que devuelve un tiempo en ms, y con esto se implementaron los temporizadores tanto del semáforo como del teclado.